Types

Lars-Henrik Eriksson*

2021-10-20

1 Introduction

So far we have not been concerned with the types of data and programs. In programming languages, type systems are used to ensure that data is used in the correct way depending on their type, e.g. that a program does not attempt to add numbers and boolean values or attempts to store a boolean value in a variable declared to hold integers.

The imperative languages **While**, **Block**, **Proc** only had a single data type (integers) so typing was not an issue. The functional language **Func** admitted multiple data types, but there was no typing of functions or identifiers. If a value had the incorrect type in a particular situation (e.g. an attempt to add a value to a pair), the semantics would simply not assign any meaning to the program. Thus **Func** could be said to have a limited form of dynamic typing. The abstract machine **AM** had a limited form of multiple data types (both integers and truth values could be on the stack) – again an attempt to use data of the incorrect type could cause the machine to become stuck.

So in a sense, both **Func** and **AM** had type checking at the point where data was actually used, but that is not what we typically mean when talking about a "typed" language.

We can distinguish between dynamic and static typing. Dynamic typing means that type checks are made at runtime. Static typing means that types are checked at compile-time. There is much more to be said about typing, but this is sufficient for the purpose of this course.

2 Typing in imperative languages

We will extend the language **Proc** so that boolean values can be stored in variables and so that variables can be declared to have a particular type (integer or boolean). The abstract syntax for boolean expressions is changed to include variables, assignment statements are changed to include boolean expressions

*Translated into LaTeX by Elias Castegren in 2024

and variable declarations will be changed to give the type of the variable being declared. All other syntactic categories of **Proc** are unchanged.

$$
\begin{array}{lcl}
b & ::= & x \mid \texttt{true} \mid \texttt{false} \mid a1 = a2 \mid a1 \leq a2 \mid \neg b \mid b1 \wedge b2 \\
S & ::= & x := a \mid x := b \mid \texttt{skip} \mid S_1; S_2 \mid \textbf{if } b \textbf{ then } S_1 \textbf{ else } S_2 \mid \\
 & & \textbf{while } b \textbf{ do } S \mid \textbf{begin } D_V\ D_P\ S \textbf{ end} \mid \textbf{call } p \\
D_V & ::= & \texttt{int}\ x := a; D_V \mid \texttt{bool}\ x := b; D_V \mid \varepsilon
\end{array}
$$

Since the syntax does not distinguish between integer and boolean variables, it becomes possible that an integer variable is used where a boolean value is required and vice versa. Proper dynamic (runtime) type checking will require the semantics to keep track of the type of a variable as well as its value so the derivation rules will also have to be modified to include the various type checks.

None of the necessary changes to the semantics are fundamentally different from what we have seen before so we will not explore dynamic typing further.

Static type checking is more interesting. We want to define under what conditions a program is correctly typed ("type correct"). Since with static typing this can be done without running the program ("at compile-time") we will use a special semantics to determine type correctness. Such a semantics which defines "compile time" properties is called a static semantics for the language. The kind of semantics that we have studied before which describes what happens when the program is run is called a dynamic semantics.

For a static semantics to be meaningful, the dynamic semantics must implement static scoping rules – otherwise there is in general no way of knowing the type of a variable without knowing the execution sequence up to the point where the variable is used. Thus we will assume that the particular dynamic semantics of **Proc** that we use is the one with fully static scoping.

To determine if a statement S is type correct, we need to know the types of the variables used in S. To keep track of this, we use a type environment. The type environment is a partial function from variable names to types, but it is customarily viewed not as a function but as a set or sequence of variable-type pairs, each of which are written $x : t$. x is a variable name and t is one of the symbols `int` or `bool`.

We will say that a statement S (arithmetic expression a or boolean expression b) is correctly typed in a type environment env, written

$$env \vdash S \quad env \vdash_a a \quad env \vdash_b b$$

The static semantics has a number of derivation rules and axiom schemas to show how we can derive these facts for each form of expression or statement.

Arithmetic expressions

$$[\text{var_}a_t] \quad env \vdash_a x \quad \text{If } x : int \in env$$

$$[\text{num}_t] \quad env \vdash_a n$$

$$[\text{op}_t] \quad \frac{env \vdash_a a_1 \qquad env \vdash_a a_2}{env \vdash_a a_1 \odot a_2} \quad \text{where } \odot \text{ is } +, - \text{ or } *$$

Boolean expressions

$$[\text{var_b}_t] \quad env \vdash_b x \quad \text{If } x : bool \in env$$

$$[\text{true}_t] \quad env \vdash_b \texttt{true}$$

$$[\text{false}_t] \quad env \vdash_b \texttt{false}$$

$$[\text{rel}_t] \quad \frac{env \vdash_a a_1 \qquad env \vdash_a a_2}{env \vdash_b a1 \odot a2} \quad \text{where } \odot \text{ is } = \text{ or } \leq$$

$$[\text{not}_t] \quad \frac{\vdash_b b}{env \vdash_b \neg b}$$

$$[\text{and}_t] \quad \frac{env \vdash_b b_1 \qquad env \vdash_b b_2}{env \vdash_b b_1 \wedge b_2}$$

Statements (assignment and control flow)

$$[\text{ass_a}_t] \quad \frac{env \vdash_a x \qquad env \vdash_a a}{env \vdash x := a}$$

$$[\text{ass_b}_t] \quad \frac{env \vdash_b x \qquad env \vdash_b b}{env \vdash x := b}$$

$$[\text{skip}_t] \quad env \vdash \texttt{skip}$$

$$[\text{comp}_t] \quad \frac{env \vdash S_1 \qquad env \vdash S_2}{env \vdash (S_1; S_2)}$$

$$[\text{if}_t] \quad \frac{env \vdash_b b \qquad env \vdash S_1 \qquad env \vdash S_2}{env \vdash \textbf{if } b \textbf{ then } S_1 \textbf{ else } S_2}$$

$$[\text{while}_t] \quad \frac{env \vdash_b b \qquad env \vdash S}{env \vdash \textbf{while } b \textbf{ do } S}$$

Statements (blocks and procedures)

$$[\text{block_va}_t] \quad \frac{env \vdash_a a \qquad env[x \mapsto \texttt{int}] \vdash \textbf{begin } D_V\ D_P\ S \textbf{ end}}{env \vdash \textbf{begin } \texttt{int}\ x := a; D_V\ D_P\ S \textbf{ end}}$$

$$[\text{block_vb}_t] \quad \frac{env \vdash_b b \qquad env[x \mapsto \texttt{bool}] \vdash \textbf{begin } D_V\ D_P\ S \textbf{ end}}{env \vdash \textbf{begin } \texttt{bool}\ x := b; D_V\ D_P\ S \textbf{ end}}$$

$$[\text{block_p}_t] \quad \frac{env \vdash S \qquad env \vdash \textbf{begin } D_P\ S' \textbf{ end}}{env \vdash \textbf{begin proc } p \textbf{ is } S; D_P\ S' \textbf{ end}}$$

$$[\text{block_S}_t] \quad \frac{env \vdash S}{env \vdash \textbf{begin } S \textbf{ end}}$$

$$[\text{call}_t] \quad env \vdash \textbf{call } p$$

The rules generally just state that a statement is type correct if all of its parts are. In the cases where a statement includes an expression, that expression must be of the appropriate type.

The rules for blocks are the most complicated ones. They are written so that if a variable is declared, then the type of that variable is entered into the type environment when type checking the rest of the block.

When type checking a program, the types of any global variables used must be included in an initial type environment, see Example T.1.

Example T.1

This **Proc** program

```
begin
  int x := y;
  proc p is x := 0 - x;
  if x ≤ 0 then call p else skip;
  y := x
end
```

is type correct, given that y is a global variable of type `int`. The following derivation is evidence. Side conditions are not shown. Rule names are removed for axioms. The placeholder tree T_1 is given below. The initial type environment is $\{y : \texttt{int}\}$.

$$[\text{block_va}_t] \quad \frac{[\text{block_p}_t] \quad \frac{[\text{ass_a}_t] \quad \{y : \texttt{int}, x : \texttt{int}\} \vdash_a x \qquad [\text{op}_t] \quad \frac{\{y : \texttt{int}, x : \texttt{int}\} \vdash_a 0 \qquad \{y : \texttt{int}, x : \texttt{int}\} \vdash_a x}{\{y : \texttt{int}, x : \texttt{int}\} \vdash_a 0 - x}}{\{y : \texttt{int}, x : \texttt{int}\} \vdash x := 0 - x} \qquad T_1}{\{y : \texttt{int}, x : \texttt{int}\} \vdash \textbf{begin proc } p \textbf{ is } x := 0 - x; \ldots \textbf{ end}} \qquad \{y : \texttt{int}\} \vdash_a y}{\{y : \texttt{int}\} \vdash \textbf{begin } \texttt{int}\ x := y; \ldots \textbf{ end}}$$

The placeholder tree T_1 shows that the body of the block is type correct:

$$
\begin{array}{rl}
[\text{block_S}_t] & \dfrac{[\text{comp}_t]\ \dfrac{T_2 \qquad [\text{ass_a}_t]\ \dfrac{\{y : \texttt{int}, x : \texttt{int}\} \vdash_a y \qquad \{y : \texttt{int}, x : \texttt{int}\} \vdash_a x}{\{y : \texttt{int}, x : \texttt{int}\} \vdash y := x}}{\{y : \texttt{int}, x : \texttt{int}\} \vdash \textbf{if } x \leq 0 \ldots ; y := x}}{\{y : \texttt{int}, x : \texttt{int}\} \vdash \textbf{begin if } x \leq 0 \ldots ; y := x \textbf{ end}}
\end{array}
$$

The placeholder tree T_2 shows that the if statement is type correct:

$$
[\text{if}_t]\ \dfrac{[\text{rel}_t]\ \dfrac{\{y : \texttt{int}, x : \texttt{int}\} \vdash_a x \qquad \{y : \texttt{int}, x : \texttt{int}\} \vdash_a 0}{\{y : \texttt{int}, x : \texttt{int}\} \vdash_b x \leq 0} \qquad \{y : \texttt{int}, x : \texttt{int}\} \vdash \textbf{call } p \qquad \{y : \texttt{int}, x : \texttt{int}\} \vdash \texttt{skip}}{\{y : \texttt{int}, x : \texttt{int}\} \vdash \textbf{if } x \leq 0 \textbf{ then call } p \textbf{ else } \texttt{skip}}
$$

□

Exercise T.2

What happens if a program uses a variable that is neither declared in a block nor included in the initial variable environment as a global variable? Is this a desired behaviour?
□

Exercise T.3

Does it matter for the static semantics of a language if the dynamic semantics is big-step or small-step?
□

Exercise T.4

In the dynamic semantics, the process of constructing a derivation top-down may not terminate (if the program loops). What about the static semantics? Can the same thing happen with the static semantics or will an attempt to construct a derivation always either succeed or fail in a finite number of steps because no inference rules/axiom schemas are applicable? Either give an example of a program for which the construction of a derivation in the static semantics will be infinite or argue (or even better – prove) that this can not happen.
□

3 Typing in functional languages

Functional languages often have a well-developed strong type system (LISP and Erlang being notable exceptions) which can be used to infer (or "compute")

the type of an expression without needing any type declarations in the program itself.

For the functional language **Func** we will construct a static semantics that not only determines if an expression is type correct, but also determines what particular type it has.

We will say that an expression E has type t in type environment env, written

$$env \vdash E : t$$

Functional languages are generally polymorphic, meaning that a function can have more than one type as long as the types follow a common pattern. To simplify the static semantics for **Func**, we will not allow polymorphism except for the pair constructor and the built-in functions `fst`, `snd`, `hd`, `tl` and ::.

Types will be represented using this abstract syntax:

$$t ::= x \mid (t_1, t_2) \mid [t] \mid t_1 \rightarrow t_2 \mid (x)t$$

The form x is the name of a simple type, such as `int` or `bool` or a type variable (see below). (t_1, t_2) is the type of a pair where the two components have types t_1 and t_2, respectively. $[t]$ is the type of a list with elements of type t. $t_1 \rightarrow t_2$ is the type of a function with argument type t_1 and value type t_2. $(x)t$ is a polymorphic type where x is a type variable in t. See example T.5.

Example T.5

The function :: for construction of lists has type $(a)((a, [a]) \rightarrow [a])$. Its argument is a pair of an element of type a and a list of elements of the same type. Its value is another list of the same type. :: can work on all types of lists – it is polymorphic – so a is a type variable.
□

The notations (t_1, t_2) and $[t]$ are used in order to build on knowledge of Haskell. In other contexts these types are typically written $t_1 * t_2$ (or $t_1 \times t_2$) and t *list* (or *list* t), respectively. The relation $\succ$ relates a polymorphic type to an instance of that type. Informally, $t' \succ t$ if t can be obtained from t' by replacing type variables with types. Note that the $\succ$ symbol is *not* the usual "greater than" symbol $>$. Formally, $\succ$ can be defined recursively by:

$t \succ t$ holds.
$(x)t' \succ t$ holds iff $t'[x \mapsto t''] \succ t$, for some type t''.
otherwise, $t' \succ t$ does not hold.

Here $t'[x \mapsto t"]$ means textual replacement of x by t'' in t'. See page 18 of the book.

Example T.6

$(a)((a,[a]) \to [a]) \succ (\texttt{int},[\texttt{int}]) \to [\texttt{int}]$ because $((a,[a]) \to [a])[a \mapsto \texttt{int}] = ((\texttt{int},[\texttt{int}] \to [\texttt{int}]) \succ (\texttt{int},[\texttt{int}]) \to [\texttt{int}]$. So :: can be used to build lists of integers.
□

Haskell also has the concept of "type classes". To keep the type system for **Func** simple we will not consider type classes here. The rules and axiom schemas of the static semantics for **Func** are:

$$[\text{num}_t] \quad env \vdash n : \texttt{int}$$

$$[\text{id}_t] \quad env \vdash x : t \quad \text{If } x : t' \in env \text{ and } t' \succ t$$

$$[\text{if}_t] \quad \frac{env \vdash E : \texttt{bool} \qquad env \vdash E_1 : t \qquad env \vdash E_2 : t}{env \vdash \textbf{if } E \textbf{ then } E_1 \textbf{ else } E_2 : t}$$

$$[\text{pair}_t] \quad \frac{env \vdash E_1 : t_1 \qquad env \vdash E_2 : t_2}{env \vdash (E_1, E_2) : (t_1, t_2)}$$

$$[\text{fun}_t] \quad \frac{env[x \mapsto t'] \vdash E : t}{env \vdash \lambda x \to E : t' \to t}$$

$$[\text{rec}_t] \quad \frac{env[x \mapsto t' \to t] \vdash E : t' \to t}{env \vdash \textbf{rec } x \to E : t' \to t}$$

$$[\text{app}_t] \quad \frac{env \vdash E_1 : t' \to t \qquad env \vdash E_2 : t'}{env \vdash E_1\ E_2 : t}$$

The $[\text{num}_t]$ rule is straightforward – any numeral has type `int`. The $[\text{id}_t]$ rule states that an identifier gets its type from the type environment – possibly after replacing type variables with types. The $[\text{pair}_t]$ rule states that a pair gets its type from the component types. The $[\text{if}_t]$ rule states that both branches of an if expression must have the same type as the whole expression and that the test must have type `bool`. The remaining rules may need more detailed explanations:

$[\text{fun}_t]$: To show that a function abstraction has type $t' \to t$ you have to show that the function body which computes the value of the function does indeed has type t. The body can use the function argument so it must be included in the type environment when the body is type checked.

$[\text{rec}_t]$: The name of the function should have the same type as the function itself.

$[\text{app}_t]$: In a function application (**call**), the argument type of the function and the type of the argument expression must be the same. The type of the function application is the same as the value type of the function. This rule is

peculiar in that when trying to construct a derivation tree top-down (starting with the typing that we want to check) we have get no hint as to what the type t' should be – we have to guess or leave a "hole" in the derivation to be filled in later.

When type checking an expression, we will need an initial typing environment env_0 with typings for the built-in functions and values.

$$
\begin{array}{rll}
env_0 = & [& [] : (a)[a], \texttt{true} : \texttt{bool}, \texttt{false} : \texttt{bool}, \\
& & = : (a)((a,a) \to \texttt{bool}), \leq : (\texttt{int}, \texttt{int}) \to \texttt{bool}, \\
& & + : (\texttt{int}, \texttt{int}) \to \texttt{int}, - : (\texttt{int}, \texttt{int}) \to \texttt{int}, \\
& & * : (\texttt{int}, \texttt{int}) \to \texttt{int}, / : (\texttt{int}, \texttt{int}) \to \texttt{int}, \\
& & \mathit{fst} : (a)(b)((a,b) \to a), \mathit{snd} : (a)(b)((a,b) \to b), \\
& & \mathit{hd} : (a)([a] \to a), \mathit{tl} : (a)([a] \to [a]), \\
& & :: \ : (a)((a,[a]) \to [a]) \\
&] &
\end{array}
$$

Note the polymorphism of the empty list [] and the functions for pairs and lists.

Example T.7

Show that $(\lambda a \to a + 1)\ 4$ has type `int`.

Side conditions are not shown. Rule names are omitted for axioms.

$$
\dfrac{\dfrac{\dfrac{env_0[a \mapsto \texttt{int}] \vdash + : (\texttt{int}, \texttt{int}) \to \texttt{int} \qquad [\text{pair}_t]\ \dfrac{env_0[a \mapsto \texttt{int}] \vdash a : \texttt{int} \qquad env_0[a \mapsto \texttt{int}] \vdash 1 : \texttt{int}}{env_0[a \mapsto \texttt{int}] \vdash (a, 1) : (\texttt{int}, \texttt{int})}}{[\text{app}_t]\ env_0[a \mapsto \texttt{int}] \vdash a + 1 : \texttt{int}}}{[\text{fun}_t]\ env_0 \vdash \lambda a \to a + 1 : \texttt{int} \to \texttt{int}} \qquad env_0 \vdash 4 : \texttt{int}}{[\text{app}_t]\ env_0 \vdash (\lambda a \to a + 1)\ 4 : \texttt{int}}
$$

□

Example T.8

Show that the function $\lambda x \to 1 :: x$ which adds the number 1 to the beginning of a list has type $[int] \to [int]$.

Side conditions are not shown in the derivation. The side condition of the instance of the $[\text{id}_t]$ schema for the :: identifier is:

$$
\begin{array}{l}
\text{If } :: : (a)((a,[a]) \to [a]) \in env_0[x \mapsto [int]] \\
\text{and } (a)((a,[a]) \to [a]) \succ (int, [int]) \to [int]
\end{array}
$$

The first part follows from the definition of env_0, for the second part see example T.6.

$$[\text{fun}_t]\ \dfrac{[\text{app}_t]\ \dfrac{env_0[x \mapsto [int]] \vdash :: : (int, [int]) \to [int] \qquad [\text{pair}_t]\ \dfrac{env_0[x \mapsto [int]] \vdash 1 : int \qquad env_0[x \mapsto [int]] \vdash x : [int]}{env_0[x \mapsto [int]] \vdash (1, x) : (int, [int])}}{env_0[x \mapsto [int]] \vdash 1 :: x : [int]}}{env_0 \vdash \lambda x \to 1 :: x : [int] \to [int]}$$

$\square$